

2026-05-07-p2-f25-plandex-plan-trees

P2-F25 — Plandex Plan Trees + Context Compaction Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Programme position: F25 is the **fifth** Phase 2 feature of CLI-Agent Fusion (after F21 Codex Approval Modes, F22 Aider Git Auto-Commit, F23 Cline Browser Tool, F24 Codex Project Memory). Task T01 advances PROGRESS.md from “Phase 2: F24 closed; F25 next candidate (brainstorm)” to “Phase 2 of CLI-Agent Fusion programme: F25 (Plandex Plan Trees) in flight” and appends an F25 evidence header to docs/improvements/07_phase_2_evidence.md (already created in F21-T01, extended by F22-T01 + F23-T01 + F24-T01).

Goal: Ship a real, end-to-end **plan tree system** for the HelixCode CLI agent. F25 ADDS a NEW internal/plantree/ package: PlanNode (ID, Title, Description, Status, Children, ParentID, Metadata), PlanTree (Root + Name + Version), FileStore (atomic JSON Save/Load/List/Delete to .helixcode/plans/<name>.json), operations (CreateTree, BranchNode, MergeNode), VerifyTree (6 structural checks), CompactTree (F01 summariser delegation). ADDS six tools.Tool implementations: plan_create, plan_branch, plan_merge, plan_list, plan_show, plan_delete. ADDS a NEW internal/commands/plan_command.go with /plan slash (list / show / compact / verify). MODIFIES cmd/cli/main.go to construct PlanStore + register tools + register slash.

Architecture: New files under helix_code/internal/plantree/ — types.go (PlanNode + PlanTree + PlanStatus + PlanTreeSummary + PlanIssue + VerifyResult + sentinels + constants + RenderTree), store.go (FileStore with Store interface for test seam), operations.go (CreateTree + BranchNode + MergeNode — pure in-memory), verify.go (VerifyTree — orphans, cycles, depth, uniqueness), compact.go (CompactTree + Summariser interface), plan_tools.go (six tools.Tool implementations). New internal/commands/plan_command.go for the /plan slash. Three existing files get small additions: cmd/cli/main.go (construct PlanStore + RegisterPlanTools + /plan registration), internal/tools/registry.go (register 6 plan tools in buildToolList), internal/commands/builtin/register.go (register /plan slash).

Tech Stack: Go 1.24, testify v1.11, google/uuid (ALREADY direct in helix_code/go.mod), F01 compression.Summariser (ALREADY shipped). **Zero new external deps** — encoding/json, os, sync, time, errors, fmt, strings, path/filepath, context, crypto/sha256 all stdlib. go mod tidy after T09 must produce no diff in either go.mod or go.sum.

Spec: docs/superpowers/specs/2026-05-07-p2-f25-plandex-plan-trees-design.md (this commit’s companion).

Working directory for go commands: helix_code/. Git from meta-repo root.

Anti-bluff smoke (full 4-term applied to F25 surface):

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" \
    internal/plantree internal/commands/plan_command.go && echo
BLUFF || echo clean
```

Must always print clean.

Anti-bluff hot zone: §5.2 of the spec — F25 can degenerate in five ways: (a) plan_create writes invalid/empty JSON; (b) plan_branch adds orphan with unresolvable ParentID; (c) plan_merge leaves parent Metadata unchanged; (d) compact reports success but byte count unchanged; (e) verify reports clean on corrupted tree. Each mapped to unit + integration + Challenge phase per spec §5.2. Plus two bonus phases: (f) plan_show output doesn't match actual tree structure; (g) shallow compaction (depth < 3) is a no-op. Challenge harness has 7 phases (A-G) with positive byte evidence.

Why this is consequential: plan trees give the agent structured, inspectable implementation tracking. Without F25, agents hold plans in opaque conversation context that disappears on restart. With F25, plans are persisted, branchable, mergeable, verifiable, and compactible — a full project-management surface. F21-24 gave the agent approvals, auto-commits, browser eyes, and persistent memory; F25 gives it a structured plan.

Task list



P2-F25-T01 — bootstrap F25 evidence section + advance PROGRESS to F25



P2-F25-T02 — internal/plantree/types.go: PlanNode + PlanTree + PlanStatus + PlanTreeSummary + PlanIssue + VerifyResult + sentinels + constants + RenderTree() (TDD)



P2-F25-T03 — internal/plantree/store.go: FileStore Save/Load/List/Delete + atomic writes + Store interface (TDD with real tempdirs)



P2-F25-T04 — internal/plantree/operations.go: CreateTree + BranchNode + MergeNode pure-tree transforms (TDD)



P2-F25-T05 — internal/plantree/verify.go: VerifyTree 6 checks (orphans, cycles, depth, uniqueness, self-parent, node count) (TDD)



P2-F25-T06 — internal/plantree/compact.go: CompactTree + Summariser interface + threshold guard + byte reduction (TDD)



P2-F25-T07 — internal/plantree/plan_tools.go: Six tools.Tool implementations with Store seam (TDD)



P2-F25-T08 — internal/commands/plan_command.go: /plan slash (list/show/compact/verify) (TDD)



P2-F25-T09 — main.go wiring + registry registration + builtin registration + integration test



P2-F25-T10 — Challenge harness 7 phases A-G + close-out + push 4 remotes non-force

Task 1: Bootstrap F25 evidence

Append F25 section header to docs/improvements/07_phase_2_evidence.md (created in F21-T01, extended by F22-T01 + F23-T01 + F24-T01) with spec SHA placeholder (will be filled by this commit). Update PROGRESS.md current focus from “Phase 2 — CLI Agent Porting (in progress); F24 COMPLETE; F25 next candidate” to “Phase 2 of CLI-Agent Fusion programme: F25 (Plandex Plan Trees) in flight”. Insert F25 task list (10 items). Verify zero new external deps:

```
cd HelixCode && grep -E "google/uuid" go.mod
# Expected: google/uuid already present.
git diff go.mod | grep -E "^\\+|^-" | grep -v "^+++\\|^---" &&
    echo "UNEXPECTED" || echo "clean"
```

Update docs/CONTINUATION.md with F25 mid-flight section (active feature pointer, Q1-Q5 summary, task table, anti-bluff hot zone).

Commit: docs(P2-F25-T01): bootstrap F25 evidence + advance PROGRESS to F25 (Plandex Plan Trees).

Task 2: types.go (TDD)

Files: internal/plantree/types.go, internal/plantree/types_test.go, internal/plantree/doc.go

Implementation:

```
// types.go
package plantree

import (
    "encoding/json"
    "errors"
    "fmt"
    "strings"
    "time"
)

// PlanStatus enum
type PlanStatus int

const (
    StatusPending      PlanStatus = iota
    StatusInProgress
    StatusCompleted
)
```

```

    StatusFailed
    StatusPruned
)

func (s PlanStatus) String() string {
    switch s {
    case StatusPending:    return "pending"
    case StatusInProgress: return "in_progress"
    case StatusCompleted:  return "completed"
    case StatusFailed:     return "failed"
    case StatusPruned:     return "pruned"
    default:               return "unknown"
    }
}

func (s PlanStatus) Marker() string {
    switch s {
    case StatusPending:    return "[ ]"
    case StatusInProgress: return "[▶]"
    case StatusCompleted:  return "[✓]"
    case StatusFailed:     return "[×]"
    case StatusPruned:     return "[×]"
    default:               return "[?]"
    }
}

func (s PlanStatus) MarshalJSON() ([]byte, error) {
    return json.Marshal(s.String())
}

func (s *PlanStatus) UnmarshalJSON(data []byte) error {
    var str string
    if err := json.Unmarshal(data, &str); err != nil {
        return err
    }
    switch str {
    case "pending":    *s = StatusPending
    case "in_progress": *s = StatusInProgress
    case "completed":  *s = StatusCompleted
    case "failed":     *s = StatusFailed
    case "pruned":     *s = StatusPruned
    default:           return fmt.Errorf("unknown plan status: %s", str)
    }
    return nil
}

// PlanNode

```

```

type PlanNode struct {
    ID          string          `json:"id"`
    Title       string          `json:"title"`
    Description string          `json:"description"`
    Status      PlanStatus      `json:"status"`
    Children    []*PlanNode     `json:"children,omitempty"`
    ParentID    string          `json:"parent_id,omitempty"`
    Metadata    map[string]string `json:"metadata,omitempty"`
    CreatedAt   time.Time       `json:"created_at"`
    UpdatedAt   time.Time       `json:"updated_at"`
}

```

// PlanTree

```

type PlanTree struct {
    Name      string      `json:"name"`
    Version   int         `json:"version"`
    Root      *PlanNode  `json:"root"`
    CreatedAt time.Time   `json:"created_at"`
    UpdatedAt time.Time   `json:"updated_at"`
}

```

// PlanTreeSummary

```

type PlanTreeSummary struct {
    Name      string      `json:"name"`
    NodeCount int         `json:"node_count"`
    RootID    string      `json:"root_id"`
    RootTitle string      `json:"root_title"`
    CreatedAt time.Time   `json:"created_at"`
    UpdatedAt time.Time   `json:"updated_at"`
}

```

// PlanIssue

```

type PlanIssueSeverity int
const (
    SeverityError   PlanIssueSeverity = iota
    SeverityWarning
)

```

```

type PlanIssue struct {
    Severity PlanIssueSeverity `json:"severity"`
    NodeID   string                    `json:"node_id,omitempty"`
    Message  string                    `json:"message"`
}

```

```

type VerifyResult struct {
    Valid  bool      `json:"valid"`
    Issues []PlanIssue `json:"issues"`
}

```

```

// Sentinel errors
var (
    ErrPlanNotFound          = errors.New("plan not found")
    ErrNodeNotFound          = errors.New("node not found in tree")
    ErrCyclicMerge           = errors.New("merge would create a
        cycle")
    ErrPlanAlreadyExists     = errors.New("plan already exists")
    ErrTooManyNodes          = errors.New("plan tree exceeds maximum
        node count")
    ErrTreeCorrupt           = errors.New("plan tree is corrupt")
)

// Constants
const (
    MaxNodes                 = 500
    MaxNodeDepth             = 20
    MaxDescriptionBytes      = 32 * 1024
    CompactThreshold         = 128 * 1024
    StorageDir               = ".helixcode/plans"
    CompactMarker            = "[c]"
)

// RenderTree renders a PlanNode as an indented text
// representation.
func RenderTree(node *PlanNode, depth int) string {
    if node == nil {
        return ""
    }
    indent := strings.Repeat(" ", depth)
    compactTag := ""
    if node.Metadata != nil && node.Metadata["compacted"] ==
        "true" {
        compactTag = " " + CompactMarker
    }
    result := fmt.Sprintf("%s%s %s (%s)%s\n", indent,
        node.Status.Marker(), node.Title, node.ID, compactTag)
    for _, child := range node.Children {
        result += RenderTree(child, depth+1)
    }
    return result
}

// CountNodes recursively counts all nodes in the tree rooted at
// node.
func CountNodes(node *PlanNode) int {
    if node == nil { return 0 }
    count := 1

```

```

    for _, child := range node.Children {
        count += CountNodes(child)
    }
    return count
}

// MaxDepth returns the maximum depth from the given node.
func MaxDepth(node *PlanNode) int {
    if node == nil { return 0 }
    maxChild := 0
    for _, child := range node.Children {
        d := MaxDepth(child)
        if d > maxChild { maxChild = d }
    }
    return maxChild + 1
}

```

Tests (types_test.go): 1. TestPlanStatus_String — table: StatusPending → “pending”, StatusCompleted → “completed”, etc. 2. TestPlanStatus_Marker — table: StatusPending → “[]”, StatusFailed → “[x]”, etc. 3. TestPlanStatus_JSONRoundtrip — each status marshals/unmarshals correctly 4. TestPlanStatus_UnmarshalJSON_Unknown — “bogus” returns error 5. TestPlanNode_Creation — new node has correct fields, zero Children 6. TestPlanTree_Creation — tree with root, name, version 7. TestRenderTree_SingleNode — single node renders with correct marker + ID 8. TestRenderTree_MultiLevel — 3-level tree renders with correct indentation 9. TestRenderTree_CompactedNode — node with Metadata[“compacted”]=“true” shows [c] marker 10. TestRenderTree_NilNode — nil returns “” 11. TestCountNodes — tree with 7 nodes returns 7 12. TestMaxDepth — 3-level tree returns 3

Commit: feat(P2-F25-T02): plantree types — PlanNode + PlanTree + PlanStatus + sentinels + constants + RenderTree (TDD).

Task 3: store.go (TDD)

Files: internal/plantree/store.go, internal/plantree/store_test.go

Implementation:

```

// store.go
type Store interface {
    Save(tree PlanTree) error
    Load(name string) (PlanTree, error)
    List() ([]PlanTreeSummary, error)
    Delete(name string) error
}

type FileStore struct {
    dir string
    mu   sync.Mutex
}

```

```

}

func NewFileStore(baseDir string) *FileStore {
    return &FileStore{dir: filepath.Join(baseDir, StorageDir)}
}

func (fs *FileStore) Save(tree PlanTree) error {
    fs.mu.Lock()
    defer fs.mu.Unlock()

    if err := os.MkdirAll(fs.dir, 0700); err != nil {
        return fmt.Errorf("create storage dir: %w", err)
    }

    data, err := json.MarshalIndent(tree, "", "  ")
    if err != nil {
        return fmt.Errorf("marshal tree: %w", err)
    }

    path := filepath.Join(fs.dir, tree.Name+".json")
    tmp := path + ".tmp"

    if err := os.WriteFile(tmp, data, 0600); err != nil {
        return fmt.Errorf("write temp file: %w", err)
    }

    if err := os.Rename(tmp, path); err != nil {
        os.Remove(tmp)
        return fmt.Errorf("atomic rename: %w", err)
    }

    return nil
}

func (fs *FileStore) Load(name string) (PlanTree, error) {
    path := filepath.Join(fs.dir, name+".json")
    data, err := os.ReadFile(path)
    if err != nil {
        if os.IsNotExist(err) {
            return PlanTree{}, ErrPlanNotFound
        }
        return PlanTree{}, fmt.Errorf("read plan file: %w", err)
    }

    var tree PlanTree
    if err := json.Unmarshal(data, &tree); err != nil {
        return PlanTree{}, fmt.Errorf("%w: %v", ErrTreeCorrupt, err)
    }
}

```



```

    return tree, nil
}

func (fs *FileStore) List() ([]PlanTreeSummary, error) {
    entries, err := os.ReadDir(fs.dir)
    if err != nil {
        if os.IsNotExist(err) {
            return nil, nil
        }
        return nil, fmt.Errorf("read storage dir: %w", err)
    }

    var summaries []PlanTreeSummary
    for _, entry := range entries {
        if entry.IsDir() || !strings.HasSuffix(entry.Name(),
            ".json") {
            continue
        }

        name := strings.TrimSuffix(entry.Name(), ".json")
        // Try to get summary from file metadata only (fast path)
        // Fall back to Load for the full summary
        tree, err := fs.Load(name)
        if err != nil {
            continue // skip corrupted files
        }
        summaries = append(summaries, tree.Summary())
    }

    return summaries, nil
}

func (fs *FileStore) Delete(name string) error {
    path := filepath.Join(fs.dir, name+".json")
    err := os.Remove(path)
    if err != nil && !os.IsNotExist(err) {
        return fmt.Errorf("delete plan file: %w", err)
    }
    return nil
}

// Helper on PlanTree
func (pt *PlanTree) Summary() PlanTreeSummary {
    return PlanTreeSummary{
        Name:      pt.Name,
        NodeCount: CountNodes(pt.Root),
        RootID:    pt.Root.ID,
    }
}

```

```

        RootTitle: pt.Root.Title,
        CreatedAt: pt.CreatedAt,
        UpdatedAt: pt.UpdatedAt,
    }
}

```

Tests (store_test.go) — ALL against real tempdir: 1. TestFileStore_SaveLoad — save tree → load → assert fields match 2. TestFileStore_AtomicWrite — save → file exists at path, no .tmp residue 3. TestFileStore_LoadNotFound — load nonexistent → ErrPlanNotFound 4. TestFileStore_LoadCorrupted — write invalid JSON → load → ErrTreeCorrupt 5. TestFileStore_List — save 3 trees → list returns 3 summaries with correct names 6. TestFileStore_ListEmpty — empty dir → empty slice, nil error 7. TestFileStore_Delete — save → delete → load → ErrPlanNotFound 8. TestFileStore_DeleteIdempotent — delete nonexistent → nil error 9. TestFileStore_FileMode — save → stat file → mode 0600 10. TestFileStore_DirectoryCreation — NewFileStore with nonexistent parent → Save creates dirs

Commit: feat(P2-F25-T03): plantree FileStore — atomic JSON Save/Load/List/Delete + Store interface (TDD real-tempdir).

Task 4: operations.go (TDD)

Files: internal/plantree/operations.go, internal/plantree/operations_test.go

Implementation:

```

func CreateTree(name, title, description string) (*PlanTree,
    error) {
    if name == "" { return nil, errors.New("plan name
        required") }
    now := time.Now().UTC()
    root := &PlanNode{
        ID:          uuid.New().String(),
        Title:        title,
        Description:  description,
        Status:       StatusPending,
        CreatedAt:   now,
        UpdatedAt:   now,
    }
    return &PlanTree{
        Name:        name,
        Version:     1,
        Root:        root,
        CreatedAt:   now,
        UpdatedAt:   now,
    }, nil
}

```

```

func BranchNode(tree *PlanTree, parentID, title, description
    string) (*PlanNode, error) {
    if tree == nil || tree.Root == nil { return nil,
        errors.New("invalid tree") }
    if CountNodes(tree.Root) >= MaxNodes { return nil,
        ErrTooManyNodes }

    parent := findNode(tree.Root, parentID)
    if parent == nil { return nil, ErrNodeNotFound }

    depth := nodeDepth(tree.Root, parentID, 1)
    if depth >= MaxNodeDepth { return nil, fmt.Errorf("max depth
        %d exceeded", MaxNodeDepth) }

    now := time.Now().UTC()
    child := &PlanNode{
        ID:          uuid.New().String(),
        Title:        title,
        Description:  description,
        Status:      StatusPending,
        ParentID:    parentID,
        CreatedAt:   now,
        UpdatedAt:   now,
    }
    parent.Children = append(parent.Children, child)
    tree.UpdatedAt = now
    return child, nil
}

func MergeNode(tree *PlanTree, childID string) (*PlanNode,
    error) {
    if tree == nil || tree.Root == nil { return nil,
        errors.New("invalid tree") }

    child := findNode(tree.Root, childID)
    if child == nil { return nil, ErrNodeNotFound }
    if child.ParentID == "" { return nil, errors.New("root node
        cannot be merged") }

    parent := findNode(tree.Root, child.ParentID)
    if parent == nil { return nil, fmt.Errorf("parent %s not
        found: %w", child.ParentID, ErrTreeCorrupt) }

    // Cycle check: following parent chain must not reach child
    if wouldCycle(tree.Root, parent.ID, childID) {
        return nil, ErrCyclicMerge
    }
}

```

```

    if parent.Metadata == nil {
        parent.Metadata = make(map[string]string)
    }
    parent.Metadata["merged_from"] = childID
    if history, ok := parent.Metadata["merged_history"]; ok &&
        history != "" {
        parent.Metadata["merged_history"] = history + "," +
            childID
    } else {
        parent.Metadata["merged_history"] = childID
    }

    child.Status = StatusCompleted
    tree.UpdatedAt = time.Now().UTC()
    return parent, nil
}

// findNode DFS search for node by ID
func findNode(node *PlanNode, id string) *PlanNode {
    if node == nil { return nil }
    if node.ID == id { return node }
    for _, child := range node.Children {
        if found := findNode(child, id); found != nil {
            return found
        }
    }
    return nil
}

// nodeDepth returns the depth of a node with the given ID
// relative to the tree root
func nodeDepth(node *PlanNode, id string, depth int) int {
    if node == nil { return 0 }
    if node.ID == id { return depth }
    for _, child := range node.Children {
        if d := nodeDepth(child, id, depth+1); d > 0 {
            return d
        }
    }
    return 0
}

// wouldCycle returns true if following parent chain from
// ancestorID reaches descendantID
func wouldCycle(node *PlanNode, ancestorID, descendantID string)
    bool {
    current := findNode(node, ancestorID)
    for current != nil && current.ParentID != "" {

```

```

        if current.ParentID == descendantID {
            return true
        }
        current = findNode(node, current.ParentID)
    }
    return false
}

```

Tests (operations_test.go): 1. TestCreateTree — create → assert root.ID non-empty, title/description match, status=StatusPending 2. TestCreateTree_EmptyName — empty name → error 3. TestBranchNode — create tree → branch under root → assert child.ParentID==root.ID, root.Children contains child 4. TestBranchNode_NotFound — branch under nonexistent ID → ErrNodeNotFound 5. TestBranchNode_MaxDepth — branch 20 levels deep → 21st branch → error 6. TestBranchNode_MaxNodes — branch until MaxNodes → next → ErrTooManyNodes 7. TestMergeNode — create → branch → merge child → parent.Metadata["merged_from"]==child.ID, child.Status==StatusCompleted 8. TestMergeNode_RootCannotMerge — merge root → error 9. TestMergeNode_CyclicMerge_Rejected — branch → child → branch grandchild → try merge grandchild into child → ok, then try create cycle → ErrCyclicMerge 10. TestMergeNode_MergeHistory — merge child1 → merge child2 → parent.Metadata["merged_history"]=="child1ID,child2ID" 11. TestFindNode — create multi-level tree → findNode returns correct node 12. TestNodeDepth — 3-level tree → depth of deepest == 3

Commit: feat(P2-F25-T04): plantree operations — CreateTree + BranchNode + MergeNode pure-tree transforms (TDD).

Task 5: verify.go (TDD)

Files: internal/plantree/verify.go, internal/plantree/verify_test.go

Implementation:

```

func VerifyTree(tree *PlanTree) VerifyResult {
    if tree == nil || tree.Root == nil {
        return VerifyResult{Valid: false, Issues:
            []PlanIssue{{Severity: SeverityError, Message: "tree is
            empty"}}}}
    }

    var issues []PlanIssue
    nodeMap := make(map[string]*PlanNode)
    buildNodeMap(tree.Root, nodeMap)

    // 1. Unique IDs
    idCount := make(map[string]int)
    countIDs(tree.Root, idCount)
    for id, count := range idCount {
        if count > 1 {

```

```

        issues = append(issues, PlanIssue{Severity:
SeverityError, NodeID: id, Message:
fmt.Sprintf("duplicate node ID appears %d times",
count)})
    }
}

// 2. Orphaned children
for _, node := range nodeMap {
    if node.ParentID != "" {
        if _, ok := nodeMap[node.ParentID]; !ok {
            issues = append(issues, PlanIssue{Severity:
SeverityError, NodeID: node.ID, Message:
fmt.Sprintf("parent %s not found (orphan)",
node.ParentID)})
        }
    }
}

// 3. Cycles
if hasCycle(tree.Root, make(map[string]bool)) {
    issues = append(issues, PlanIssue{Severity:
SeverityError, Message: "plan tree contains a cycle"})
}

// 4. Depth limit
if depth := MaxDepth(tree.Root); depth > MaxNodeDepth {
    issues = append(issues, PlanIssue{Severity:
SeverityError, Message: fmt.Sprintf("maximum depth %d
exceeds limit %d", depth, MaxNodeDepth)})
}

// 5. Node count
if count := CountNodes(tree.Root); count > MaxNodes {
    issues = append(issues, PlanIssue{Severity:
SeverityError, Message: fmt.Sprintf("node count %d
exceeds maximum %d", count, MaxNodes)})
}

// 6. Self-parenting
for _, node := range nodeMap {
    if node.ParentID == node.ID {
        issues = append(issues, PlanIssue{Severity:
SeverityError, NodeID: node.ID, Message: "node parent
references itself"})
    }
}

```

```

    valid := true
    for _, issue := range issues {
        if issue.Severity == SeverityError {
            valid = false
            break
        }
    }

    return VerifyResult{Valid: valid, Issues: issues}
}

func buildNodeMap(node *PlanNode, m map[string]*PlanNode) {
    if node == nil { return }
    m[node.ID] = node
    for _, child := range node.Children {
        buildNodeMap(child, m)
    }
}

func countIDs(node *PlanNode, m map[string]int) {
    if node == nil { return }
    m[node.ID]++
    for _, child := range node.Children {
        countIDs(child, m)
    }
}

func hasCycle(node *PlanNode, visited map[string]bool) bool {
    if node == nil { return false }
    if visited[node.ID] { return true }
    visited[node.ID] = true
    for _, child := range node.Children {
        if hasCycle(child, visited) { return true }
    }
    visited[node.ID] = false
    return false
}

```

Tests (verify_test.go): 1. TestVerifyTree_Clean — 3-level valid tree → Valid=true, 0 issues 2. TestVerifyTree_Orphan — create orphan ref (set ParentID to bogus) → Valid=false, orphan issue found 3. TestVerifyTree_Cycle — create cycle (A child of B child of C child of A) → Valid=false, cycle issue 4. TestVerifyTree_DepthOverflow — tree at depth 21 → Valid=false, depth issue 5. TestVerifyTree_DuplicateIDs — two nodes with same ID → Valid=false, dup issue 6. TestVerifyTree_SelfParenting — node.ParentID == node.ID → Valid=false, self-parent issue 7. TestVerifyTree_NodeCountOverflow — tree with 501 nodes → Valid=false, count issue 8. TestVerifyTree_NilTree — nil → Valid=false, “tree is empty” 9. TestVerifyTree_NilRoot — tree with nil Root → Valid=false

Commit: feat(P2-F25-T05): plantree VerifyTree – 6 structural checks (orphans, cycles, depth, uniqueness, self-parent, count) (TDD).

Task 6: compact.go (TDD)

Files: internal/plantree/compact.go, internal/plantree/compact_test.go

Implementation:

```
type Summariser interface {
    Summarise(text string, maxWords int) (string, error)
}

type CompactResult struct {
    Tree          PlanTree
    OriginalBytes  int
    NewBytes       int
    NodesCompacted int
}

func CompactTree(tree *PlanTree, summariser Summariser)
    (CompactResult, error) {
    if tree == nil || tree.Root == nil {
        return CompactResult{}, errors.New("invalid tree")
    }

    origJSON, err := json.Marshal(tree)
    if err != nil {
        return CompactResult{}, fmt.Errorf("marshal: %w", err)
    }
    originalBytes := len(origJSON)

    if originalBytes < CompactThreshold {
        return CompactResult{Tree: *tree, OriginalBytes:
            originalBytes, NewBytes: originalBytes, NodesCompacted:
            0}, nil
    }

    compacted := 0
    walkAndCompact(tree.Root, 1, summariser, &compacted)

    newJSON, err := json.Marshal(tree)
    if err != nil {
        return CompactResult{}, fmt.Errorf("marshal after
        compact: %w", err)
    }
}
```



```

    return CompactResult{
        Tree:          *tree,
        OriginalBytes:  originalBytes,
        NewBytes:       len(newJSON),
        NodesCompacted: compacted,
    }, nil
}

func walkAndCompact(node *PlanNode, depth int, summariser
    Summariser, compacted *int) {
    if node == nil { return }

    for _, child := range node.Children {
        walkAndCompact(child, depth+1, summariser, compacted)
    }

    // Only compact Completed or Failed leaf nodes at depth >= 3
    if depth >= 3 && len(node.Children) == 0 && (node.Status ==
        StatusCompleted || node.Status == StatusFailed) {
        summary, err := summariser.Summarise(node.Description,
            50)
        if err == nil && summary != "" {
            if node.Metadata == nil {
                node.Metadata = make(map[string]string)
            }
            node.Metadata["compacted_bytes"] = fmt.Sprintf("%d",
                len(node.Description))
            node.Metadata["compacted"] = "true"
            node.Description = summary
            *compacted++
        }
    }
}

```

Tests (compact_test.go): 1. TestCompactTree_BelowThreshold — small tree (< CompactThreshold) → CompactResult.NodesCompacted==0, bytes unchanged 2. TestCompactTree_AboveThreshold_Compacts — create a tree with 30+ Completed leaf nodes and large descriptions → compact → NodesCompacted ≥ 1, bytes reduced 3. TestCompactTree_ByteReduction — byte diff (OriginalBytes - NewBytes) ≥ 30% of OriginalBytes 4. TestCompactTree_SummariserCalled — mock summariser counts calls → assert called exactly N times (N = compactible nodes) 5. TestCompactTree_ShallowNodesNotCompacted — Completed leaves at depth 1 or 2 → not compacted 6. TestCompactTree_PendingNodesNotCompacted — leaf at depth 3+ with StatusPending → not compacted 7. TestCompactTree_CompactedMarker — after compact, node.Metadata["compacted"]=="true" 8. TestCompactTree_SummariseError_Graceful — summariser returns error → node not compacted, no panic 9. TestCompactTree_NilInput — nil tree → error 10. TestCompactTree_PreservesRoot — after compact, root.Title/Description unchanged

Commit: feat(P2-F25-T06): plantree CompactTree + Summariser adapter
– F01 delegation + threshold guard (TDD).

Task 7: plan_tools.go (TDD)

Files: internal/plantree/plan_tools.go, internal/plantree/plan_tools_test.go

Implementation: Six tools.Tool implementations. Each tool holds a Store interface for testability. Tools call the operations package (CreateTree, BranchNode, MergeNode) for in-memory transforms then Store.Save(tree) for persistence.

```
type PlanCreateTool struct{ store Store }
type PlanBranchTool struct{ store Store }
type PlanMergeTool struct{ store Store }
type PlanListTool struct{ store Store }
type PlanShowTool struct{ store Store }
type PlanDeleteTool struct{ store Store }

// All implement tools.Tool:
//   Name() string
//   Description() string
//   Parameters() map[string]interface{}
//   Execute(ctx context.Context, params map[string]interface{})
//       (map[string]interface{}, error)
//   Category() tools.ToolCategory
//   RequiresApproval() tools.ApprovalLevel
```

Tool names: plan_create, plan_branch, plan_merge, plan_list, plan_show, plan_delete.

Category: tools.CategoryPlan (new constant added to registry.go).

Tests (plan_tools_test.go): 1. TestPlanCreateTool — execute with name/title/description → file saved, summary returned 2. TestPlanCreateTool_DuplicateName — create same name twice → second returns ErrPlanAlreadyExists 3. TestPlanBranchTool — create plan → branch → child returned with correct ParentID 4. TestPlanBranchTool_ParentNotFound → ErrNodeNotFound 5. TestPlanMergeTool → branch → merge → parent Metadata updated, child Completed 6. TestPlanListTool → create 2 plans → list returns 2 summaries 7. TestPlanShowTool → create 3-level tree → show output contains markers + IDs 8. TestPlanDeleteTool → create → delete → load → ErrPlanNotFound 9. TestPlanDeleteTool_NotFound → delete nonexistent → ErrPlanNotFound 10. TestAllTools_RequiresApproval — each tool's RequiresApproval() returns correct level 11. TestAllTools_CategoryName — each tool reports Name() + Category()

Commit: feat(P2-F25-T07): plan_tools.go — six tools.Tool implementations (plan_create/branch/merge/list/show/delete) (TDD).

Task 8: /plan slash command (TDD)

Files: internal/commands/plan_command.go, internal/commands/plan_command_test.go

Implementation:

```
type PlanCommand struct {
    store      plantree.Store
    summariser plantree.Summariser
}

// Implements commands.Command:
// Name() string → "plan"
// Aliases() []string → ["pl"]
// Description() string → "Manage plan trees"
// Usage() string → "/plan [list|show <name>|compact <name>|
//     verify <name>]"
// Execute(ctx context.Context, cmdCtx *CommandContext)
//     (string, error)
```

Subcommand routing: no-args → list; list → tabular output; show <name> [--id <node-id>] → RenderTree; compact <name> → CompactTree + report byte reduction; verify <name> → VerifyTree + issue listing.

Tests (plan_command_test.go): 1. TestPlanCommand_List → empty store → “No plans found” 2. TestPlanCommand_List_WithPlans → 2 plans → output contains both names 3. TestPlanCommand_Show → create → show → output contains markers + node ID 4. TestPlanCommand_Show_NotFound → nonexistent → “plan not found” 5. TestPlanCommand_Show_Subtree → show with -id → only subtree rendered 6. TestPlanCommand_Compact → compact calls summariser, byte reduction reported 7. TestPlanCommand_Verify_Clean → verify → “valid” 8. TestPlanCommand_Verify_Corrupt → corrupt tree → verify shows issues 9. TestPlanCommand_DefaultToList → no-args → calls list 10. TestPlanCommand_HelpText → help output contains subcommands

Commit: feat(P2-F25-T08): /plan slash command (list/show/compact/verify) (TDD).

Task 9: main.go wiring + registry + integration test

Files modified: - internal/tools/registry.go — add CategoryPlan const + register 6 plan tools in buildToolList - internal/commands/builtin/register.go — register /plan slash - cmd/cli/main.go — construct FileStore + summariser → RegisterPlanTools → /plan registration - tests/integration/cmd/p2f25_wiring_test.go (NEW)

Integration test: 1. Construct FileStore in tempdir 2. Construct PlanCommand 3. Execute plan_create tool → assert file created 4. Execute plan_branch tool → assert child added 5. Execute plan_list tool → assert 1 plan listed 6. Execute plan_show tool → assert tree rendered 7.

Execute plan_merge tool → assert parent metadata updated 8. Execute plan_delete tool → assert file removed 9. Execute plan_verify → assert clean 10. Assert anti-bluff clean on all new files

Commit: feat(P2-F25-T09): main.go wiring + CategoryPlan + RegisterPlanTools + /plan builtin + integration test.

Task 10: Challenge harness + close-out

Challenge harness directory: tests/integration/cmd/p2f25_challenge/

7 phases (A-G):

PHASE-A (create + verify schema): 1. Call plan_create with name="challenge-plan", title="Challenge Plan", description="A plan for the F25 challenge" 2. Read .helixcode/plans/challenge-plan.json from disk 3. Assert: valid JSON, root.id is non-empty UUID, root.title == "Challenge Plan", root.status == "pending" 4. Assert: node count == 1, root has 0 children

PHASE-B (branch + structural integrity): 1. Call plan_branch 3 times under root: "Task 1", "Task 2", "Task 3" 2. Read JSON file 3. Assert: root.Children array has length 3 4. Verify: all 3 children have ParentID == root.ID 5. Verify: 0 orphans (VerifyTree returns Valid=true)

PHASE-C (merge + metadata): 1. Call plan_merge on child "Task 1" 2. Read JSON file 3. Assert: child Status == "completed" 4. Assert: root.Metadata["merged_from"] == task1.ID 5. Assert: root.Metadata["merged_history"] == task1.ID

PHASE-D (compact + byte reduction): 1. Create deep tree with 30+ Completed leaf nodes (each with 500-char descriptions) 2. Record serialized byte size (pre) 3. Call CompactTree with F01 summariser 4. Record serialized byte size (post) 5. Assert: post < pre (reduction ≥ 30%) 6. Assert: root.Title + root.Description unchanged 7. Assert: at least one node has Metadata["compacted"]="true"

PHASE-E (verify detects corruption): 1. Create 3 deliberately corrupted plan JSON files: a. corrupt-orphan.json — child node's ParentID references nonexistent node b. corrupt-cycle.json — A→B→C→A cycle c. corrupt-dup.json — two nodes with identical ID 2. Run VerifyTree on each 3. Assert: all 3 return Valid=false 4. Assert: each has at least 1 issue with Severity=Error 5. Assert: orphan → issue mentions "orphan"; cycle → issue mentions "cycle"; dup → issue mentions "duplicate"

PHASE-F (show output structural match): 1. Create known 3-level tree: Root (pending) |— Child1 (in_progress) | |— Grandchild1 (pending) |— Child2 (completed) 2. Call plan_show 3. Assert: output contains "[] Root" at indent 0 4. Assert: output contains "[▶] Child1" at indent 2 5. Assert: output contains "[] Grandchild1" at indent 4 6. Assert: output contains "[✓] Child2" at indent 2 7. Assert: node IDs appear in parentheses after each title

PHASE-G (shallow no-compact guard): 1. Create tree with all Completed leaves at depth 1 and 2 2. Call CompactTree 3. Assert: NodesCompacted == 0 4. Assert: bytes unchanged

Close-out: - Anti-bluff smoke clean - Cross-compile linux/amd64 PASS - All 10 plan tasks ticked - PROGRESS.md updated: "F25 COMPLETE; F26 next candidate" - CONTINUATION.md F25 mid-flight section marked CLOSED - Push 4 meta-repo remotes non-force - Push Challenges submodule (for challenge harness)

Verification at T09

```
cd HelixCode
go build ./internal/plantree/... ./internal/commands/... ./cmd/
    cli/...
go test -count=1 -race ./internal/plantree/
go test -count=1 -race ./internal/commands/
go test -count=1 -tags=integration -run
    TestPlanTree_Integration ./tests/integration/
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder"
    internal/plantree internal/commands/plan_command.go &&
    echo BLUFF || echo clean
```

Plan written. Execute via subagent-driven-development starting with T01.